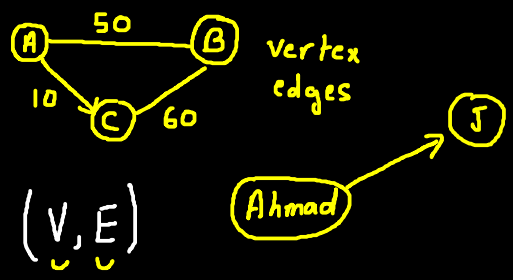
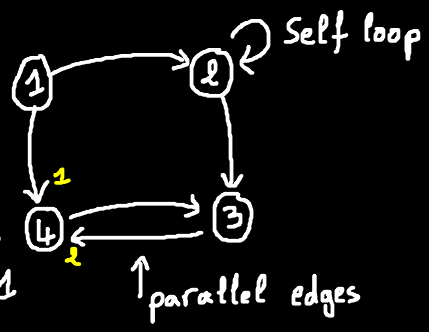
Graphs



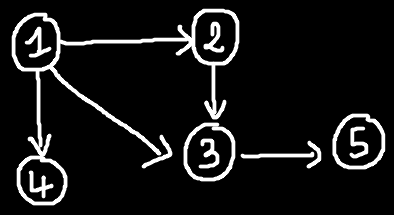
vertex
edges

$G = (V, E)$

Directed Graph :

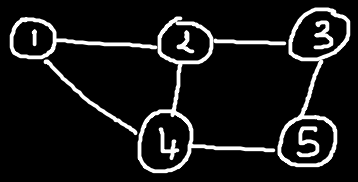


Indegree : 2
out degree : 1



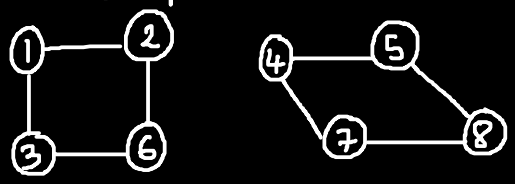
Simple Directed Graph
(no self loop + no parallel edgs)

degree(3) = 2

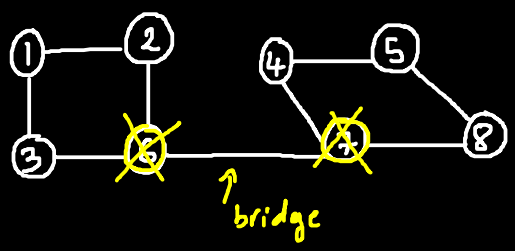


Undirected Graph

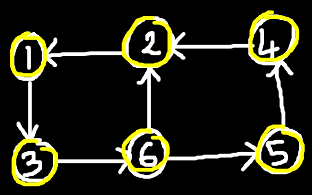
2 components



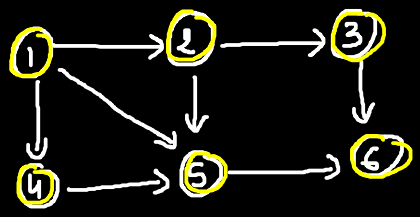
non-connected Graph



6 and 7 are articulation points

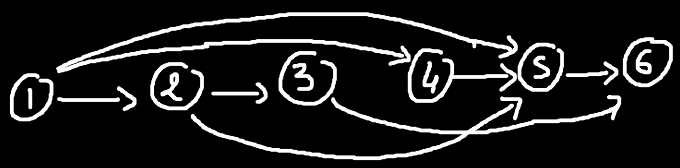


Strongly connected



Directed Asyclic Graph
(DAG)

Topological ordering



Adjacency matrix

$$A = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 1 & 1 \\ 2 & 1 & 0 & 0 & 1 \\ 3 & 1 & 0 & 0 & 1 \\ 4 & 1 & 1 & 1 & 0 \end{array}$$

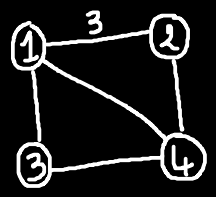
$|V|^2$

$O(N^2)$

self loops

4x4

$$A[i][j] = \begin{cases} 1 & (i,j) \\ 0 & \end{cases}$$



$G = (V, E)$

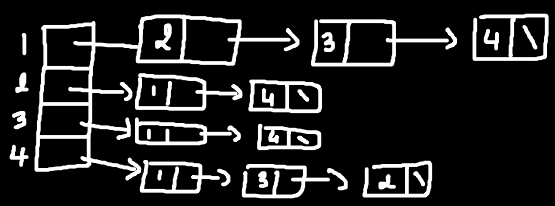
$|V| = 4$

$|E| = 5$

undirected $A[i,j] = A[j,i]$

row column

Adjacency list



$|V| + 2 \times |E|$

$O(|V| + |E|)$

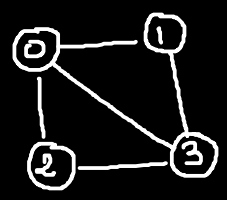
```
struct node {
    int vertex;
    int weight;
    struct node * next;
};
```

Compact list

5	8	10	12	15	1	2	3	0	3	0	3	0	1	2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Vertices

$\underbrace{|V|}_{4} + \underbrace{2|E|}_{10} + 1 = 15$

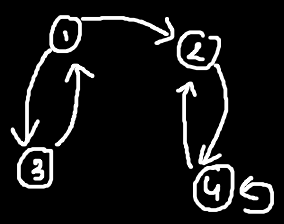


$O(|V| + 2|E|) = O(|V| + |E|)$ $O(n)$ $(n = |V|)$

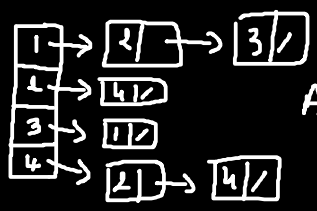
Representation of directed Graphs

$$G = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 1 & 0 \\ 2 & 0 & 0 & 0 & 1 \\ 3 & 1 & 0 & 0 & 0 \\ 4 & 0 & 1 & 0 & 1 \end{array}$$

$G[i,j] \neq G[j,i]$

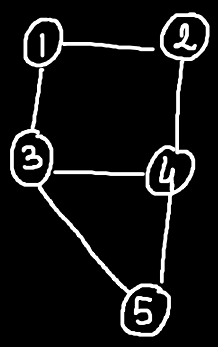


$G =$



Adjacency list

BFS : Breadth First Search

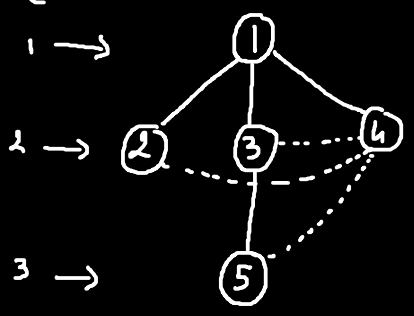


What nodes are reachable from node x?
What is the shortest path?
Does a cycle exist?
Is the graph connected?

→ Traversal Algorithms

$G = (V, E)$

(level order traversal)



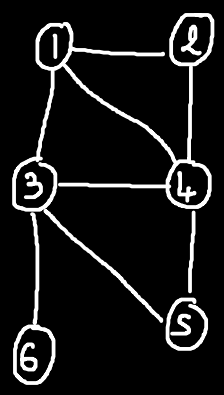
Source = 5

BFS : 5, 3, 4, 1, 2

BFS : 5, 4, 3, 2, 1

BFS : 1, 2, 3, 4, 5

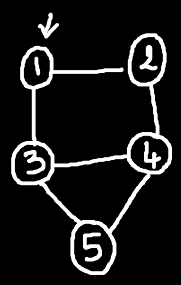
{ Visiting
Exploring



BFS : 1, 2, 3, 4, 5, 6

Queue : ~~1~~, ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~

BFS



G

$$A = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 & 0 \\ 3 & 1 & 1 & 0 & 1 & 1 \\ 4 & 0 & 1 & 1 & 0 & 1 \\ 5 & 0 & 0 & 1 & 1 & 0 \end{array}$$

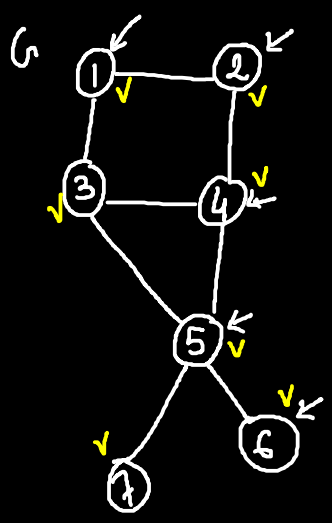
visited :	1	1	1	1	1

Queue :	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~

```
void BFS (int source)
{
    cout << source;
    visited[source] = 1;
    enqueue(source);
    while (!q.IsEmpty)
        u = dequeue();
        for(int v=1; v<=n; v++) if( A[u][v]==1 && !visited[v])
                                    visited[v] = 1;
                                    enqueue(v);
    }
```

4×3

DFS: Depth First Search



$$A = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 & 0 \\ 3 & 1 & 1 & 0 & 1 & 1 \\ 4 & 0 & 1 & 1 & 0 & 1 \\ 5 & 0 & 0 & 1 & 1 & 0 \end{array}$$

Stack

~~1~~	~~2~~	~~4~~	~~5~~	~~6~~	~~5~~	

Visited

1	1	1	1	1	1	1

DFS : 1, 2, 4, 5, 6, 7

```
DFS (int u)
{
    if (visited[u] == 0)
    {
        cout << u;
        visited[u] = 1;
        for (v=1 ; v<=n ; v++)
        {
            if(A[u][v] == 1 &&
                visited[v] == 0)
                DFS(v);
        }
    }
}
```

Island

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

visited = $\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & & & & \\ 0 & & 0 & & \\ 0 & & & & \end{pmatrix}$

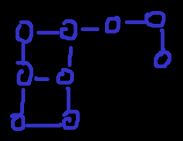
nIlands

1 -> for

2 -> Run BFS

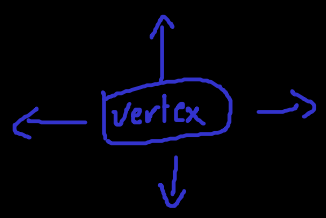
3 -> increment Ilands

Graph



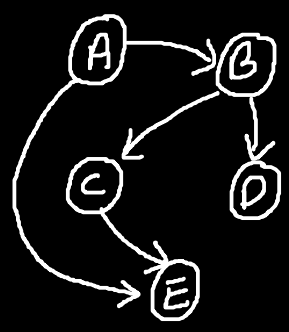
2 components

Directions



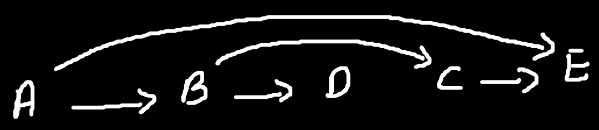
Topological Sorting

$G = (V, E)$

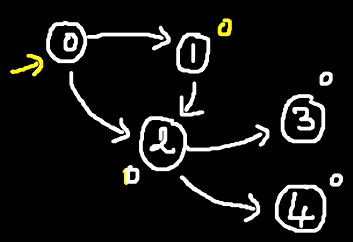


DAG

linear ordering of all vertices such that for each (u, v)
u comes before v



Kahn's algorithm



$$A = \begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 & 1 & 1 \\ 3 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 \end{array}$$

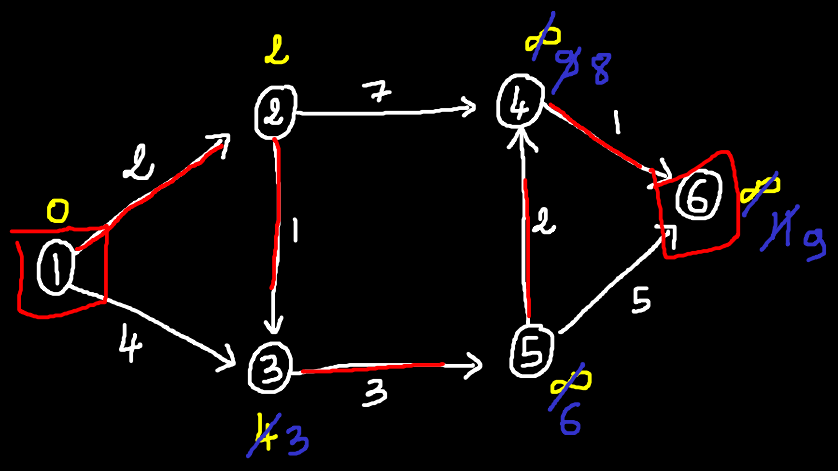
indegrees $\quad 0 \quad 1 \quad 2 \quad 1 \quad 1$

1 - Find all vertices with indegree 0
2 - enqueue
3 - dequeue + add it to the answer
4 - reduce indegree of neighbors

queue $\quad \boxed{\cancel{0}} \boxed{\cancel{1}} \boxed{\cancel{2}} \boxed{\cancel{3}} \boxed{\cancel{4}}$

Topo $\quad 0, 1, 2, 3, 4$

Dijkstra Algorithm



$|V| \times |V|$

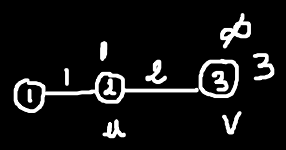
Time complexity

$O(|V|^2)$

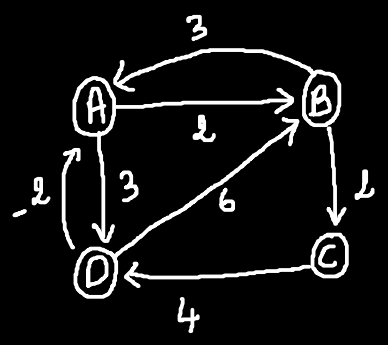
Relaxation

if $(d[u] + c(u,v) < d[v])$

$d[v] = d[u] + c(u,v)$



Floyd Warshall Algorithm



$$M = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 2 & \infty & 3 \\ B & 3 & 0 & 2 & \infty \\ C & \infty & \infty & 0 & 4 \\ D & -2 & 6 & \infty & 0 \end{array}$$

Goal:

$$\begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 2 & \infty & 3 \\ B & 3 & 0 & 6 & \\ C & \infty & & 0 & \\ D & & & & 0 \end{array}$$

(Shortest distances)

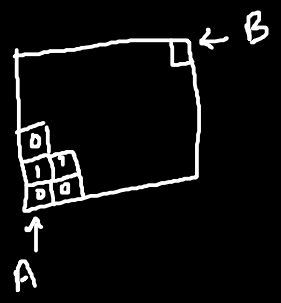
matrix distance

$dist[i][j]$ $\quad$ $dist[i][k] + dist[k][j]$

```
for K=1 to V:
    for i=1 to V:
        for j=1 to V:
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j]
```

A^*

$O(E)$



Geometric case

Heuristic

$$\sqrt{\Delta y^2 + \Delta x^2}$$

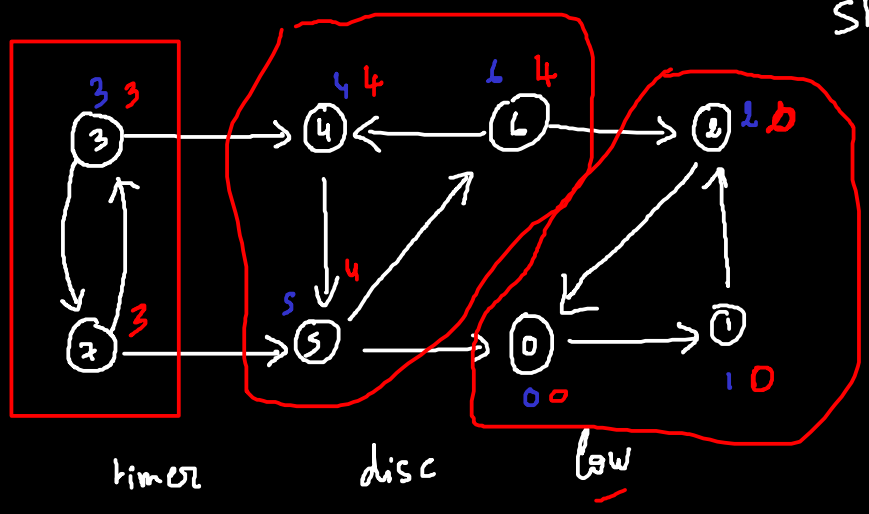
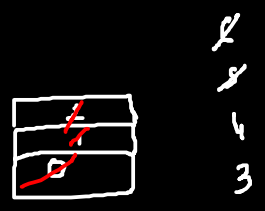
$$h = \Delta y^2 + \Delta x^2$$

$O(V^2)$ V: vertices

$O(V.\log V)$

Tarjan Algorithm

Strongly connected



Stack